Ejercicios de C++ para practicar

Colección de pequeños ejercicios con pruebas automáticas para practicar las técnicas de C++ que necesitarás en las prácticas de la asignatura: paso de parámetros, strings, ficheros, vectores, clases, punteros y listas enlazadas.

> **No es necesario hacerlas todas.** Son un banco de ejercicios de apoyo, no una tarea obligatoria ni evaluable. Haz solo las que te resulten interesantes:
>
> - las que te sirvan para **practicar** algo que todavía no dominas,
> - las que te ayuden a **resolver una duda** concreta,
> - las que te enseñen a **afrontar las dificultades** que te vas encontrando en las prácticas (por ejemplo, un fallo de memoria o un `current` que queda colgando).
>
> Si ya controlas un tema, sáltatelo. Si una práctica se te atasca, busca aquí el grupo que trabaja justo esa dificultad y haz solo esos ejercicios.

Contenido de la carpeta

```
src/          aquí escribes tus soluciones (un esqueleto por ejercicio)
tests/        las pruebas (no las modifiques) y tests.csv con los grupos
run_tests.sh  script que compila y ejecuta las pruebas
Makefile      solo sirve para "make clean"
```

Cada ejercicio tiene un código de grupo y número, por ejemplo **B03**, y estos ficheros:

Fichero	Para qué sirve
`src/B03_string_find.cc`	esqueleto con la descripción del ejercicio: **aquí escribes tu solución**
`tests/test_B03_string_find.cc`	la prueba que comprueba tu solución

En los ejercicios de clases (grupos E, F y G) el esqueleto es una **carpeta** (`src/E03_class_copy_ctor/`) con la cabecera (`.h`, ya escrita) y el fichero `.cc` que tienes que completar. Cada ejercicio trae su propia copia de la clase, con solo los métodos que necesita: no busques en otro ejercicio el método que te toca escribir, porque no está.

El grupo G es distinto: la lista la escribes entera

En los grupos A-F cada ejercicio te deja un hueco que rellenar. En el grupo **G** no hay nada hecho: el `List.cc` llega **vacío** y la lista enlazada la construyes tú desde el primer nodo. Lo único que se da es el `List.h` (la declaración de la clase, que es lo que usan las pruebas) y la clase `Node`, igual que en el enunciado de la práctica 2.

Los ocho ejercicios son **acumulativos**: cada uno añade un método a la clase del anterior. La forma de trabajarlos es:

1. Haz G01 hasta que salga **PASS**.
2. **Copia tu `List.cc`** a la carpeta de G02 y añade solo el método nuevo.
3. Repite hasta G08.

Puedes empezar por cualquier ejercicio, pero entonces tendrás que escribir también los métodos de los anteriores (están declarados en su `List.h`). Mientras falte alguno, la prueba da **ERROR** y el mensaje del enlazador dice exactamente cuál: `referencia a 'List::push_front(int)' sin definir`.

Progresión

Los grupos siguen un orden de menor a mayor dificultad, y cada grupo se apoya en los anteriores. Dentro de cada grupo los ejercicios también van ordenados.

Grupo	Tema	De qué va
A	Paso por valor y por referencia	Cuándo una función cambia (o no) las variables de quien la llama: `int`, `double` y `string`; `&` y `const&`
B	Strings	Recorrer y construir strings; buscar, contar, convertir con `stoi` y dar formato `m:ss`
C	Entrada/salida	`stringstream`, `getline`, leer y escribir ficheros
D	Vectores	Añadir, buscar, insertar, borrar, filtrar y ordenar; `vector<string>`
E	Clases	Constructores, `static`, copia, destructor, `operator=`, `<`, `==`, `<<`; clases que contienen otras
F	Punteros básicos	`&`, `*`, `->`, `nullptr`, `new` y `delete`
G	Listas enlazadas	Construir una lista simplemente enlazada **desde cero**: insertar, recorrer, buscar, vaciar, borrar y copia profunda

Dentro de un grupo, cada ejercicio añade poco a lo anterior. Si un ejercicio se te

resiste, prueba el anterior del mismo grupo.

Cómo se trabaja con un ejercicio

1. Lee el esqueleto (`src/...`). Empieza con un bloque de comentarios que explica: qué debe hacer la función, una idea de solución y varios ejemplos de entrada y salida.
2. Si aparece una línea `// TODO: incluir la librería <...>`, escribe ese `#include` en ese sitio.
3. Escribe tu solución donde indica `// ESCRIBE TU FUNCIÓN AQUÍ` o en el método marcado con `// TODO: implementar`. En el grupo G no hay marcas: escribes todos los métodos que declara el `List.h` del ejercicio.
4. Ejecuta la prueba del ejercicio (ver abajo) y corrige hasta que salga **PASS**.

En algunos ejercicios de strings y de paso de parámetros el esqueleto **no trae la función**, solo su nombre, parámetros y lo que devuelve. Tú decides cómo declarar los parámetros (por valor, por referencia, constantes o no): es parte del ejercicio. Mientras la función no exista, la prueba dará **ERROR** de compilación.

Ejecutar las pruebas

```
./run_tests.sh          # todos los grupos
./run_tests.sh B        # solo el grupo B
./run_tests.sh B03      # solo el ejercicio B03
```

Al terminar se muestra un informe con un resultado por ejercicio:

Resultado	Significado
PASS	la prueba se cumple
FAIL	el programa funciona pero devuelve un resultado distinto del esperado
ERROR	no compila (o falta la función): mira el mensaje del compilador
CRASH	el programa se ha caído (acceso a memoria inválida, doble `delete`...)
TIMEOUT	tarda demasiado (posible bucle infinito)
MEMORY	la prueba puede pasar, pero `valgrind` detecta fugas o accesos inválidos

`MEMORY` solo aparece si tienes `valgrind` instalado (`sudo apt install valgrind`). Es muy útil en los grupos E, F y G.

El detalle de cada ejecución está en `tests/output/`:

- `test_XNN_....test.txt`: el informe de la prueba. Te dice qué comprobación falló, qué valor esperaba y cuál obtuvo.

- `test_XNN_....cout.txt`: todo lo que tu código escribe con `cout`. Así puedes imprimir mensajes de depuración sin que se mezclen con el informe.
- `*.valgrind.txt.<número>`: el informe de `valgrind`, si detectó algún error.

Cada ejecución del script **borra `tests/output/`** y lo vuelve a generar.

Consejos

- **Dibuja antes de programar** en los grupos F y G. Casi todos los errores con punteros se evitan dibujando cajas y flechas.
- En el grupo G, antes de pasar al ejercicio siguiente asegúrate de que el anterior sale **PASS** y sin **MEMORY**: vas a arrastrar ese código hasta G08.
- Prueba siempre los casos extremos: vacío, un solo elemento, el primero, el último.
- No cambies los **nombres ni las firmas** que indica el esqueleto: las pruebas los usan tal cual.
- No modifiques los ficheros de `tests/` (salvo si haces pruebas nuevas en el grupo Z).
- Si una prueba te parece incorrecta o el enunciado ambiguo, avisa al profesorado: puede que haya que mejorarlo.